UNIVERSIDAD INTERAMERICANA DE PUERTO RICO · RECINTO DE FAJARDO

PROYECTO FINAL · DESARROLLO WEB FULL-STACK

Sabías que PR

Evidencia de pruebas

Fase 4: Pruebas y publicación · pruebas funcionales, corrección de errores y publicación en línea

Estudiante	William R. Collado
Número de estudiante	M00105850
Curso	COMP2053 — Desarrollo Web Full-Stack
Profesora	Milagros Donato Cintrón
Institución	Universidad Interamericana de Puerto Rico, Recinto de Fajardo
Repositorio	github.com/Wcollado72/SabiasquePR
Sitio en línea	sabiasquepr.com
Fecha	24 de octubre de 2026

1. Resumen

Este documento reúne la evidencia de las pruebas de **Sabías que PR**: las pruebas automáticas del backend (Jest y Supertest), la verificación del sitio y del API publicados desde fuera, y las pruebas de carga y de pantalla en navegadores. Todas las cifras se **leen de los resultados guardados** por los scripts del repositorio (ver la sección 7); nada se escribió a mano. Lo que no se pudo probar se dice con claridad en la sección 5.

85/85	1261/1261	93,0 %	33 s
suites de pruebas pasan	pruebas pasan (0 fallan, 0 omitidas)	cobertura de líneas (2248 de 2418)	duración de la ejecución completa

Ejecución del 24/10/2026 11:00 con Node v24.13.0. Comando: `cd backend && npx jest --coverage`.

Cobertura de código

Cobertura global: **93,0 %** de líneas, 91,8 % de sentencias, 92,6 % de funciones y 86,3 % de ramas, medida sobre `backend/src/**` excepto `server.js`. Por carpeta:

Carpeta	Archivos	Líneas	Sentencias	Funciones	Ramas
`src`	2	96,6 %	96,7 %	77,8 %	92,9 %
`src/lib`	1	100,0 %	100,0 %	100,0 %	100,0 %
`src/middleware`	1	94,7 %	87,7 %	100,0 %	84,0 %
`src/routes`	17	93,6 %	92,6 %	95,0 %	88,7 %
`src/services`	18	87,0 %	85,8 %	88,5 %	75,9 %
`src/test-utils`	1	89,5 %	90,9 %	100,0 %	100,0 %
`src/util`	9	97,7 %	94,3 %	100,0 %	86,7 %
`src/validators`	17	99,8 %	99,0 %	100,0 %	97,3 %

Las carpetas de rutas y validadores, donde viven el CRUD y la validación, superan el 90 % de líneas. La carpeta más baja es `src/services` (87 % de líneas): son los agentes de IA (Investigador, Redactor, Traductor) y el extractor de artículos, que dependen de la API de Claude y de páginas externas; sus pruebas cubren la lógica interna con servicios simulados, no las llamadas reales.

Qué significan y qué no significan estas pruebas

Las pruebas **no necesitan red ni base de datos real**: ninguna se conectó a Supabase, a Stripe ni a la API de Claude. Las pruebas de integración de las rutas usan Supertest sobre la aplicación Express real, pero sustituyen la base de datos (`jest.mock('../db')`) y la verificación del token (un doble de prueba, `test-utils/authMock.js`) para simular usuarios y roles. En las pruebas del middleware real (`auth*.test.js`) la librería `jose` también está simulada: comprueban que el servidor exige emisor y audiencia y responde 401 ante un token inválido, no que Supabase firme un token. Por eso la sección 4 repite las comprobaciones clave **contra el servidor real en producción**. El esquema de la base de datos se valida con `npm run db:probar-schema`, que sí necesita una base de datos y **no se ejecutó para este documento**.

2. Suites de pruebas por área

El proyecto tiene **85 archivos de pruebas** con **1261 pruebas**. Se agrupan por área según su ubicación en el código (rutas, validadores, middleware, scripts, etc.).

Área / archivo de pruebas	Pruebas	Estado
Autenticación y roles — 5 suites	**56**	
src/middleware/auth.jwt.test.js	8	✓
src/middleware/auth.mfa.test.js	16	✓
src/middleware/auth.test.js	5	✓
src/routes/perfil.integration.test.js	18	✓
src/util/edad.test.js	9	✓
CRUD: noticias, contenidos, productos y debate — 13 suites	**277**	
src/clasificacion.test.js	9	✓
src/regiones.test.js	14	✓
src/routes/agradecimientos.integration.test.js	17	✓
src/routes/comentarios.integration.test.js	21	✓
src/routes/contenidos.integration.test.js	50	✓
src/routes/foro.integration.test.js	65	✓
src/routes/fuentes.integration.test.js	14	✓
src/routes/newsletter.integration.test.js	5	✓
src/routes/noticias.integration.test.js	45	✓
src/routes/ordenes.integration.test.js	2	✓
src/routes/productos.integration.test.js	13	✓
src/routes/reportes.integration.test.js	20	✓
src/util/autorVisible.test.js	2	✓
Validadores de datos y utilidades — 15 suites	**213**	
src/util/deteccionRiesgo.test.js	12	✓
src/util/slugify.test.js	5	✓
src/validators/agradecimiento.test.js	16	✓
src/validators/comentario.test.js	7	✓
src/validators/contenido.test.js	37	✓
src/validators/fechaNacimiento.test.js	22	✓
src/validators/foroPost.test.js	22	✓
src/validators/fuentes.test.js	13	✓
src/validators/gastoServicio.test.js	9	✓
src/validators/iaHerramientas.test.js	21	✓
src/validators/noticia.test.js	23	✓
src/validators/perfil.test.js	5	✓
src/validators/producto.test.js	9	✓
src/validators/reporte.test.js	5	✓
src/validators/traduccion.test.js	7	✓
Pagos Stripe y webhook — 2 suites	**19**	
src/routes/stripe.test.js	10	✓
src/routes/stripeWebhook.test.js	9	✓

Seguridad y cabeceras — 3 suites	**20**	
src/app.express5.test.js	10	✓
src/app.hardening.test.js	6	✓
src/app.ratelimit.test.js	4	✓
Panel de administración, agentes de IA y traducciones — 30 suites	**421**	
src/correccion.test.js	13	✓
src/ia-herramientas.test.js	11	✓
src/routes/admin.gastos.integration.test.js	17	✓
src/routes/admin.integration.test.js	41	✓
src/routes/admin.traducciones.integration.test.js	21	✓
src/routes/brigada.integration.test.js	37	✓
src/routes/contenidoIa.integration.test.js	10	✓
src/routes/correccion.integration.test.js	20	✓
src/routes/semillas.e3.test.js	15	✓
src/routes/semillas.integration.test.js	25	✓
src/services/articleExtractor.test.js	4	✓
src/services/cazadora.test.js	16	✓
src/services/clasificadora.test.js	14	✓
src/services/contrapunto.test.js	7	✓
src/services/corrector.test.js	8	✓
src/services/detective.test.js	11	✓
src/services/farolera.test.js	13	✓
src/services/grieta.test.js	9	✓
src/services/guardiana.test.js	13	✓
src/services/investigador.test.js	13	✓
src/services/investigadorVarias.test.js	7	✓
src/services/metadatosFuente.test.js	16	✓
src/services/pintora.test.js	7	✓
src/services/redactor.test.js	7	✓
src/services/resumidor.test.js	4	✓
src/services/traductor.test.js	11	✓
src/taller.test.js	26	✓
src/util/semillaAContenido.test.js	11	✓
src/util/traducciones.test.js	8	✓
src/util/usoIa.test.js	6	✓

Frontend, i18n y build — 12 suites	**226**	
`src/build.legal-fecha.test.js`	2	✓
`src/frontend-apoyo.test.js`	39	✓
`src/frontend-auth-errores.test.js`	11	✓
`src/frontend-brigada.test.js`	56	✓
`src/frontend-contacto.test.js`	18	✓
`src/frontend-contenido-ia.test.js`	10	✓
`src/frontend-fase6.test.js`	12	✓
`src/frontend-filtros.test.js`	26	✓
`src/frontend-fuentes.test.js`	12	✓
`src/frontend-panel-arranque.test.js`	1	✓
`src/frontend-panel-ux.test.js`	25	✓
`src/i18n/i18n.test.js`	14	✓
Scripts de base de datos, respaldo y migración — 4 suites	**20**	
`scripts/aplicar-plantillas-correo.test.js`	4	✓
`scripts/lib/delta.test.js`	7	✓
`scripts/lib/env-ensayo.test.js`	4	✓
`scripts/lib/filtrar-toc.test.js`	5	✓
Configuración de despliegue — 1 suites	**9**	
`src/netlify-config.test.js`	9	✓

Total: 85 suites, 1261 pruebas. Nota: no hay una suite de Jest dedicada al esquema de la base de datos; esa verificación es el script `db:probar-schema` *mencionado arriba.*

3. Qué demuestran las pruebas respecto a la rúbrica

Cada fila cita pruebas reales, con su nombre tal como aparece en el resultado de Jest y el archivo donde viven. El generador de este documento falla si una prueba citada no existe en el resultado, de modo que no se puede citar una prueba inexistente.

Requisito	Qué se comprueba	Pruebas concretas

CRUD completo	Crear, leer, editar y borrar (noticias; igual en contenidos, productos y publicaciones del debate), con 404 si no existe.	`POST /api/noticias 201 al crear una noticia valida como admin`
		src/routes/noticias.integration.test.js · pasa
		`GET /api/noticias devuelve la lista de noticias`
		src/routes/noticias.integration.test.js · pasa
		`PUT /api/noticias/:id 200 al editar una noticia existente`
		src/routes/noticias.integration.test.js · pasa
		`DELETE /api/noticias/:id 200 al eliminar una noticia existente`
		src/routes/noticias.integration.test.js · pasa

JWT verificado en el servidor	El servidor exige emisor y audiencia del token y rechaza uno inválido con 401 (con jose simulado; ver el recuadro de la sección 1).	`verificacion del JWT con emisor y audiencia requireAuth exige iss = <proyecto>/auth/v1 y aud = authenticated`
		src/middleware/auth.jwt.test.js · pasa
		`verificacion del JWT con emisor y audiencia un token con emisor/audiencia equivocados (jose lo rechaza) responde 401`
		src/middleware/auth.jwt.test.js · pasa
		`requireAuth responde 401 si no hay encabezado Authorization`
		src/middleware/auth.test.js · pasa
		`admin con sesion aal1 (solo contrasena) pierde el poder: rol efectivo "usuario" y requireAdmin responde 403 mfa_required`
		src/middleware/auth.mfa.test.js · pasa

Requisito	Qué se comprueba	Pruebas concretas

Rutas protegidas: 401 y 403	401 sin sesión y 403 si la sesión no tiene el rol requerido, en rutas de escritura y del panel.	`POST /api/noticias 401 sin autenticacion`
		src/routes/noticias.integration.test.js · pasa
		`POST /api/noticias 403 si el usuario no es admin`
		src/routes/noticias.integration.test.js · pasa
		`GET /api/admin/estado-seguridad 403 si el usuario no es admin ni moderador`
		src/routes/admin.integration.test.js · pasa
		`DELETE /api/foro/posts/:id (permisos) 403 si el usuario no es el autor ni admin`
		src/routes/foro.integration.test.js · pasa

Validación de datos	Los datos inválidos se rechazan con 400 antes de tocar la base de datos, incluido el bloqueo de enlaces peligrosos.	`validateNoticia rechaza titulo muy corto`
		src/validators/noticia.test.js · pasa
		`POST /api/noticias 400 si el cuerpo es invalido`
		src/routes/noticias.integration.test.js · pasa
		`validatePost rechaza esquemas peligrosos (javascript:, data:) en el enlace multimedia`
		src/validators/foroPost.test.js · pasa
		`POST /api/productos 400 si el precio es invalido`
		src/routes/productos.integration.test.js · pasa

Pagos y despliegue	El webhook de Stripe rechaza firmas falsas; la configuración de Netlify oculta los archivos internos.	`POST /api/stripe/webhook 400 si la firma es invalida`
		src/routes/stripeWebhook.test.js · pasa
		`netlify.toml el archivo interno "/build.js" se devuelve como 404`
		src/netlify-config.test.js · pasa

4. Verificación en producción desde fuera

El 24/10/2026 11:00 se ejecutaron, con `curl` desde esta computadora, 22 peticiones contra el API publicado (`sabiasquepr-backend-uufz.onrender.com`) y el sitio (`sabiasquepr.com`). Son solo lecturas públicas y peticiones **sin credenciales válidas** que el servidor debe rechazar; no crean ni modifican datos. Resultado: **22 de 22 dieron el código esperado**. La respuesta se muestra literal (recortada a 70 caracteres).

4.1 Lecturas públicas y rutas protegidas

Petición	Esperado	Obtenido	Tiempo	Respuesta literal
Lecturas públicas (deben dar 200 con datos)				
`GET /api/health`	200	**200** ✓	0,58 s	`{"ok":true,"hora_servidor":"2026-10-24T15:00:09.095Z"}`
`GET /api/noticias`	200	**200** ✓	0,38 s	`{"ok":true,"noticias":[{"id":"la-nasa-halla-el-crater-mas-grande-forma`
`GET /api/contenidos?seccion=educacion`	200	**200** ✓	0,38 s	`{"ok":true,"contenidos":[]}`
`GET /api/contenidos?seccion=revolucion-humana`	200	**200** ✓	0,34 s	`{"ok":true,"contenidos":[{"id":"recorrido-en-kart-por-una-via-costera-`
`GET /api/productos`	200	**200** ✓	0,36 s	`{"ok":true,"productos":[{"id":"bolso-puerta-san-juan","nombre":"Bolso`
Rutas protegidas SIN token (deben dar 401)				
`POST /api/noticias`	401	**401** ✓	0,29 s	`{"ok":false,"error":"Falta token de autenticacion."}`
`PUT /api/noticias/x`	401	**401** ✓	0,30 s	`{"ok":false,"error":"Falta token de autenticacion."}`

Petición	Esperado	Obtenido	Tiempo	Respuesta literal
`DELETE /api/noticias/x`	401	**401** ✓	0,26 s	`{"ok":false,"error":"Falta token de autenticacion."}`
`GET /api/admin/gastos`	401	**401** ✓	0,27 s	`{"ok":false,"error":"Falta token de autenticacion."}`
`GET /api/admin/uso-ia`	401	**401** ✓	0,27 s	`{"ok":false,"error":"Falta token de autenticacion."}`
`POST /api/foro/posts`	401	**401** ✓	0,28 s	`{"ok":false,"error":"Falta token de autenticacion."}`
`GET /api/me`	401	**401** ✓	0,49 s	`{"ok":false,"error":"Falta token de autenticacion."}`
Rutas protegidas con un token INVENTADO (deben dar 401)				
`POST /api/noticias`	401	**401** ✓	1,35 s	`{"ok":false,"error":"Token invalido o expirado."}`
`GET /api/admin/gastos`	401	**401** ✓	0,27 s	`{"ok":false,"error":"Token invalido o expirado."}`
`GET /api/me`	401	**401** ✓	0,27 s	`{"ok":false,"error":"Token invalido o expirado."}`
Webhook de Stripe sin firma válida (debe dar 400)				
`POST /api/stripe/webhook`	400	**400** ✓	0,29 s	`{"ok":false,"error":"Firma de webhook invalida: No stripe-signature he`

Datos reales devueltos: 3 noticias, 11 contenidos de Revolución Humana y 7 productos. El token inventado es un JWT con firma falsa: `eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJpbnZlbnRhZG8ifQ.firma-falsa`.

Observación: Educación no tiene contenidos publicados todavía

`GET /api/contenidos?seccion=educacion` responde 200 pero con la lista vacía (`{"ok":true,"contenidos":[]}`). No es un error del servidor: aún no hay piezas de Educación publicadas en la base de datos de producción, y la página lo muestra con su mensaje de «Aún no hay contenido». Conviene publicar contenido en esa sección antes de la demostración si se quiere mostrar esa ruta con datos.

4.2 Sitio estático (Netlify)

Comprobación	Esperado	Obtenido	Detalle
`GET sabiasquepr.com/build.js`	404	**404** ✓	build.js interno no se publica
`GET sabiasquepr.com/package.json`	404	**404** ✓	package.json interno no se publica
`HEAD http://sabiasquepr.com/`	301	**301** ✓	redirección a HTTPS → https://sabiasquepr.com/
`HEAD fabulous-tartufo-fcb839.netlify.app/`	301	**301** ✓	redirección al dominio propio → https://sabiasquepr.com/

Cabeceras de seguridad recibidas en `GET https://sabiasquepr.com/` (`curl -sI`):

Cabecera	Valor recibido (recortado)
`content-security-policy`	✓ `default-src 'self'; script-src 'self' 'sha256-MfSipPWpHRRfpUtYkfE+dcFrhFT7Tl76ce1KQoSs29g=' 'sha256-TZVou…`
`strict-transport-security`	✓ `max-age=63072000; includeSubDomains; preload`
`x-frame-options`	✓ `DENY`
`x-content-type-options`	✓ `nosniff`
`referrer-policy`	✓ `strict-origin-when-cross-origin`
`permissions-policy`	✓ `camera=(), microphone=(), geolocation=()`

El API también manda sus cabeceras de endurecimiento (helmet): `content-security-policy`, `strict-transport-security`, `x-frame-options`, `x-content-type-options`, `referrer-policy`, `cross-origin-resource-policy`. No aparece `x-powered-by` (no anuncia el framework). Además responde con límite de pedidos (`ratelimit-limit: 300`).

Tiempos: todas las respuestas del API llegaron en menos de 1,5 s porque el servicio estaba despierto. En el nivel gratuito de Render el primer pedido tras un rato de inactividad puede tardar alrededor de 50 s (arranque en frío); esa espera no se midió en esta ejecución.

5. Pruebas en navegadores y dispositivos

Qué se probó automáticamente. Se cargaron en **sabiasquepr.com** las siete páginas principales (inicio, noticias, Revolución Humana, educación, tienda, apoyo/premium y contacto) con cada navegador instalado en esta computadora, sin pantalla, controlado por el protocolo DevTools, en tres tamaños de pantalla: escritorio (1280×800), tableta (768×1024) y móvil (390×844, con emulación táctil). Por cada combinación se verificó: (1) que el documento responde 2xx, dispara «load» y muestra contenido; (2) que **no hay desbordamiento horizontal** (`scrollWidth ≤ innerWidth`); y (3) que no hay errores en la consola.

Límites de esta prueba (léase antes de las tablas)

Solo se probaron **Google Chrome 155.0.8059.85, Microsoft Edge 156.0.4314.42**, ambos con motor Chromium, en Windows. Los tamaños de tableta y móvil son **emulación de pantalla en un navegador de escritorio**, no teléfonos ni tabletas reales. **No se probó** Firefox (no está instalado en esta computadora), Safari en un iPhone, iPad o Mac reales, Chrome en Android ni Samsung Internet: esas pruebas están en la tabla de pruebas manuales pendientes de más abajo y **no deben darse por hechas**. Safari sí se probó con su **motor (WebKit)** corriendo en Windows: ver la sección 5.4; eso reduce el riesgo, pero **no sustituye** probar en un iPhone o un Mac reales.

5.1 Google Chrome 155.0.8059.85

Página	Escritorio carga · sin desborde · sin errores	Tableta carga · sin desborde · sin errores	Móvil carga · sin desborde · sin errores
Inicio	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
Noticias	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
Revolución Humana	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
Educación	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
Tienda	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
Premium (apoyo)	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
Contacto	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓

5.2 Microsoft Edge 156.0.4314.42

Página	Escritorio carga · sin desborde · sin errores	Tableta carga · sin desborde · sin errores	Móvil carga · sin desborde · sin errores
Inicio	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
Noticias	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
Revolución Humana	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
Educación	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
Tienda	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
Premium (apoyo)	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓
Contacto	✓ ✓ ✓	✓ ✓ ✓	✓ ✓ ✓

Medidas de ancho (contenido / ventana, en píxeles)

Un desbordamiento horizontal existiría si el ancho del contenido fuera mayor que el de la ventana. Valores medidos con `document.documentElement.scrollWidth` y `window.innerWidth`:

Página	Chrome Escritorio	Chrome Tableta	Chrome Móvil	Edge Escritorio	Edge Tableta	Edge Móvil
Inicio	1280 / 1280	768 / 768	390 / 390	1280 / 1280	768 / 768	390 / 390
Noticias	1280 / 1280	768 / 768	390 / 390	1280 / 1280	768 / 768	390 / 390
Revolución Humana	1280 / 1280	768 / 768	390 / 390	1280 / 1280	768 / 768	390 / 390
Educación	1280 / 1280	768 / 768	390 / 390	1280 / 1280	768 / 768	390 / 390
Tienda	1280 / 1280	768 / 768	390 / 390	1280 / 1280	768 / 768	390 / 390
Premium (apoyo)	1280 / 1280	768 / 768	390 / 390	1280 / 1280	768 / 768	390 / 390
Contacto	1280 / 1280	768 / 768	390 / 390	1280 / 1280	768 / 768	390 / 390

Resultado: de 42 combinaciones (navegador × tamaño × página), 42 cargaron correctamente, 42 no tienen desbordamiento horizontal y 42 no tienen errores de consola propios. Respuestas de red con código 400 o mayor durante las cargas: 0.

Hallazgo: la consola muestra avisos de CSP que no vienen del código del proyecto

En 42 de 42 cargas la consola registra 2 avisos de seguridad (CSP) cuyo origen es `about:srcdoc`: un `<iframe id="nl-badge-frame" title="Powered by Netlify">` que Netlify añade al sitio y cuyo código en línea la política CSP del sitio bloquea (uno por estilo y otro por script). Esos avisos **no se contaron como errores propios** en las tablas, pero se declaran aquí con su cifra completa. El código del repositorio no contiene ese iframe (búsqueda de «srcdoc» en `frontend/`: sin resultados). No afecta el funcionamiento observado; conviene revisar en Netlify si ese distintivo es intencional. Ejemplo literal: `security: Applying inline style violates the following Content Security Policy directive 'style-src 'self' https://fonts.googleapis.com'. Either the '…`

En una ejecución previa de este mismo script apareció además, una sola vez (Edge, escritorio, Tienda), la excepción «Identifier 'AssistantLoadState' has already been declared»; ese nombre no existe en el código del proyecto y no se repitió en la ejecución final, así que se atribuye a un script interno del navegador, no al sitio.

5.4 Motor de Safari (WebKit) en iPhone, iPad y Mac

Qué se probó. Con **WebKit 18.2 (Playwright)** (el motor de Safari) se cargaron en **sabiasquepr.com** las mismas siete páginas en tres tamaños con el agente de usuario de Safari: **iPhone** (390×844, táctil), **iPad** (820×1180, táctil) y **Mac** (1440×900). Por cada combinación se midió: que responda 200, que no haya errores de JavaScript, que no haya desbordamiento horizontal, que ningún campo de formulario tenga letra menor de 16 px (con menos, Safari de iPhone hace zoom solo al tocar el campo) y que no haya respuestas de red con código 400 o mayor. Además se repitió con el **almacenamiento bloqueado**, como Safari con «Bloquear todas las cookies» (tocar `localStorage` lanza `SecurityError`).

Medición (21 combinaciones: 3 dispositivos × 7 páginas)	Antes de los ajustes (2026-10-24)	Después de los ajustes (2026-10-24)
Responden 200	21 de 21	21 de 21
Sin errores de JavaScript	21 de 21	21 de 21
Sin desbordamiento horizontal	21 de 21	21 de 21
Campos de formulario de 16 px o más (sin zoom en iPhone)	12 de 21	21 de 21
Sin respuestas de red con código 400 o mayor	21 de 21	21 de 21
Con el almacenamiento bloqueado: sin errores de JavaScript	0 de 21	21 de 21

Hallazgos de esta prueba y cómo se corrigieron

1. Zoom al tocar un campo (iPhone). Los campos de texto medían 15,2 px; en 9 de 21 combinaciones había campos por debajo de 16 px (inicio, educación y contacto). Se subió la regla base a 1rem (16 px). **2. Carrito.** El cajón del carrito usaba `100vh`, que en iPhone incluye la barra de Safari y puede tapar el botón de pagar; se agregó `100dvh` con `100vh` de respaldo. **3. Almacenamiento bloqueado.** Con «Bloquear todas las cookies», 21 de 21 combinaciones lanzaban un error de JavaScript sin atrapar (el tema claro/oscuro y la confirmación de edad de las secciones 18+ leían `localStorage`/`sessionStorage` sin protección); ahora 0. Las tres correcciones se publicaron y la medición «después» se hizo contra producción. Los avisos de CSP del distintivo «Powered by Netlify» (un iframe ajeno al código del proyecto) se separan, igual que en Chrome y Edge.

Límite de esta prueba

Es el **motor** de Safari ejecutándose en una computadora con Windows, controlado de forma automática. No es un iPhone, un iPad ni un Mac reales: no cubre el teclado en pantalla, los gestos, la barra de Safari que sube y baja, ni las diferencias propias de cada versión de iOS o macOS. Por eso la prueba manual en un iPhone real (modelo del dueño del proyecto: iPhone 14) sigue pendiente en la sección 5.5.

5.5 Pruebas manuales pendientes (por completar)

Estas pruebas NO se han hecho. Requieren dispositivos o navegadores que no se pudieron usar de forma automática. Quien las realice marca cada casilla con ✓ o ✗ y anota el modelo del dispositivo, la versión del navegador y la fecha. Todas se hacen abriendo `https://sabiasquepr.com`.

Clave	Qué verificar en cada dispositivo
A	Carga y se ve completa: Inicio y Noticias (sin cortes ni texto montado).
B	No hay barra de desplazamiento horizontal en ninguna página (se arrastra el dedo de lado y no se mueve).

C	El menú se abre y se cierra y cada enlace lleva a su página.
D	Iniciar sesión: el cuadro se abre, se puede escribir y cerrarlo.
E	El selector ES / EN cambia el idioma de la página.
F	Tienda y Apoyo: los productos y los botones se ven completos.
G	Contacto: el formulario se ve y se puede escribir en sus campos.

Dispositivo y navegador	A	B	C	D	E	F	G	Modelo, versión y fecha
iPhone 14 — Safari (iOS)	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	Resultado: _____ _____
Android — Chrome	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	Resultado: _____ _____
Samsung Galaxy — Samsung Internet	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	Resultado: _____ _____
Firefox en escritorio (no instalado aquí)	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	Resultado: _____ _____
Tableta real (iPad o Android)	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	Resultado: _____ _____
Safari en Mac (si hay acceso a una)	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	Resultado: _____ _____

*Estado actual de esta tabla: **todas las casillas están vacías a propósito**; ninguna de estas pruebas se ha realizado.*

6. Errores encontrados y corregidos durante el proyecto

Lista tomada del historial real de git (el commit, la fecha y el título se leen del repositorio al generar este documento; la descripción resume el mensaje del propio commit). Muestra el ciclo de probar, encontrar el error y corregirlo; no es una lista exhaustiva.

Commit	Fecha	Problema encontrado	Corrección
`ceb7bc2`	2026-09-23	Inicio de sesión y registro: la tecla Enter no hacía nada (los campos no estaban en un formulario) y, en modo registro, un listener que no se quitaba lanzaba inicio de sesión y registro a la vez.	Se unificó en un solo manejador que revisa el modo actual y soporta Enter.
`3f23ca8`	2026-09-23	Noticias: el campo «destacado» se infería por posición en un arreglo (mismo defecto que ya se habia corregido en contenidos).	Se pasa de forma explícita; pruebas nuevas «noticia destacada» (147/147 en ese momento). También se corrigió una vulnerabilidad moderada de una dependencia (npm audit fix).
`b024dca`	2026-09-23	La conexión a Postgres no tenía límite de tiempo: un despliegue con una conexión mala se quedaba colgado muchos minutos en vez de fallar rápido.	Se añadieron tiempos límite de conexión y de consulta en backend/src/db.js.
`10e797e`	2026-09-24	Netlify no aplicaba la redirección de «/» porque existe un index.html físico en esa ruta.	Se marcó force=true en la regla de netlify.toml.
`b37d840`	2026-09-24	Menú de escritorio: «Hazte premium» se desbordaba sobre el botón de tema cuando el correo del usuario era largo.	Se recortó el correo mostrado y el menú pasó a flex-wrap; el commit indica que se verificó en el navegador a 1440 px.

Commit	Fecha	Problema encontrado	Corrección
`e794e0b`	2026-09-26	Auditoría de seguridad: enlaces javascript:/data: en el debate y en contenidos (XSS almacenado), conexión a la BD sin verificar certificado y errores 500 que filtraban mensajes de Postgres.	Validadores que bloquean esos esquemas (con pruebas), SSL con certificado fijado, errores genéricos y CSP en netlify.toml.
`6994f0d`	2026-09-26	Fix urgente: la CSP recién añadida bloqueaba cdn.jsdelivr.net y rompía el inicio de sesión en producción (detectado en el sitio real).	Se añadió el dominio a script-src de netlify.toml.
`047f18b`	2026-09-27	Panel de administración: tras cerrar sesión el panel seguía visible con los datos ya cargados.	admin.js escucha onAuthStateChange y vuelve a tapar el panel; además se limpiaron opciones obsoletas del campo Región en validadores y schema.sql.
`50b90a2`	2026-10-02	Endurecimiento: el API anunciaba Express, el JWT no exigía emisor/audiencia, las cuentas suspendidas seguían válidas y /api/health devolvía el mensaje interno de error.	helmet, jwtVerify con issuer y audience, banned_until, JSON malformado → 400 y cuerpo grande → 413; 14 pruebas nuevas (app.hardening.test.js y auth.jwt.test.js).
`6a435c2`	2026-10-20	Contacto: la nota del correo se comportaba mal tras enviar el formulario; además había 5 textos sin traducir (nombres propios).	Corregido junto con la clasificación de contenidos; el commit solo reporta pruebas en verde (694 en ese momento; la cifra actual está en la sección 1).
`a72918f`	2026-10-20	Educación: el banner de reglas del foro mostraba la etiqueta de edad «12+», que una política sin etiqueta de edad (COPPA) evita porque puede leerse como «dirigido a menores».	Las reglas del foro pasan a ser para todo público, sin etiqueta de edad (se quitó el badge «12+» de las páginas), con prueba nueva en frontend-filtros.test.js.
`ddcd338`	2026-10-20	Privacidad: las publicaciones y comentarios del debate devolvían y mostraban el correo del autor (autor_email).	La API devuelve autor_nombre (nombre del perfil o un alias genérico «Participante-xxxx» derivado del id, util/autorVisible.js); pruebas de que la API pública no selecciona el correo.
`1de54a9`	2026-10-23	Panel de administración y debate: la selección de texto no era visible en los campos oscuros del panel, no había aviso del campo obligatorio que faltaba y la tecla Enter enviaba el formulario; en el debate no existía el botón Editar.	Selección de texto visible, aviso con apertura del panel del primer campo obligatorio que falta, Enter ya no envía, botón Editar (PATCH) en el debate (además, cuadro propio para Reportar); el commit reporta 713 pruebas en verde en ese momento.

7. Cómo ejecutar las pruebas

Todos los comandos se escriben en una terminal (PowerShell o Git Bash) con Node 22 o superior instalado.

Para qué	Comando
Instalar dependencias del backend (una vez)	`cd backend; npm install`
Ejecutar todas las suites de pruebas	`cd backend; npm test`
Ejecutar las pruebas con informe de cobertura	`cd backend; npm run test:coverage`
Revisar el estilo del código (ESLint)	`cd backend; npm run lint`
Guardar los resultados de Jest y la cobertura (docs/entrega/evidencia-pruebas.json)	`cd backend; node scripts/recoger-evidencia.js`
Repetir la verificación contra producción (usa curl)	`cd backend; node scripts/verificar-produccion.js`
Probar Chrome y Edge en tres tamaños de pantalla	`cd backend; node scripts/probar-navegadores.js`
Probar el motor de Safari (WebKit; Playwright se instala aparte, ver el encabezado del script)	`cd backend; PLAYWRIGHT_DIR=../playwright-webkit node scripts/probar-webkit.js`
Regenerar este PDF, desde la raíz del repositorio (con --recoger repite también las tres mediciones)	`python docs/generate_evidencia_pruebas.py`

Los resultados temporales (informes de Jest y de cobertura) se guardan en la carpeta temporal del sistema, por lo que no ensucian el repositorio; la carpeta `backend/coverage/` está ignorada por git.

8. Hallazgos y pendientes

Resumen honesto de lo que esta evidencia deja abierto. Ninguna de estas filas se ha resuelto ni se da por probada.

Tema	Qué se encontró	Estado
Educación sin contenido	La ruta de contenidos de Educación responde 200 con lista vacía en producción (sección 4).	Pendiente: publicar piezas en esa sección antes de la demostración.
Avisos de CSP en consola	42 de 42 cargas muestran 2 avisos del iframe «Powered by Netlify» bloqueado por la CSP (sección 5).	Pendiente: revisar en Netlify si el distintivo es intencional.
Dispositivos y navegadores no probados	Firefox, Safari en un iPhone/iPad/Mac reales, Chrome en Android y Samsung Internet no se probaron en dispositivos reales; las tabletas y móviles automáticos son emulación. Safari sí se probó con su motor WebKit en Windows (sección 5.4).	Pendiente: completar la tabla de la sección 5.5 (casillas vacías).
Cobertura de los servicios de IA	src/services tiene 87 % de líneas cubiertas; las llamadas reales a la API de Claude no se prueban de forma automática.	Aceptado: se prueba la lógica con servicios simulados.
Esquema de la base de datos	El script db:probar-schema necesita una base de datos y no se ejecutó para este documento.	Pendiente: ejecutarlo y anotar el resultado si se quiere incluir como evidencia.
Arranque en frío del API	Render (nivel gratuito) puede tardar unos 50 s tras inactividad; en esta ejecución el servicio ya estaba despierto y no se midió.	Informativo: el flujo keep-alive del repositorio mitiga la espera.

Conclusión. Con lo medido en esta ejecución: 1261 de 1261 pruebas automáticas pasan, 22 de 22 comprobaciones en producción dieron el resultado esperado y 42 de 42 cargas en Chrome y Edge fueron correctas y sin desbordamiento horizontal. Lo anterior respalda el CRUD, las rutas protegidas, la validación de datos y la publicación en línea; no sustituye las pruebas manuales en dispositivos reales listadas arriba.